Cosma — SSDLC Supporting Evidence

Application: **Cosma** (Zoom Marketplace submission) · Vendor: **Cosmaneura** · Prepared: 2026-04-20

> This document provides supporting evidence that Cosma follows a secure software development lifecycle (SSDLC). Every artifact below is taken directly from the project's source repository; file paths are shown in each section so a reviewer can verify any claim against the same source of truth.

1. Source control & peer code review

EVIDENCE All source code is versioned in a private GitHub repository. Changes reach production only through pull requests targeting the protected `master` branch, reviewed by a peer engineer before merge. `staging` and `master` are the only branches that trigger deploys (see §3).

Control	Implementation
Version control	GitHub (private repo)
Branch strategy	Feature branches → PR → `master` (prod) / `staging`
Pull-request review	Required before merge; history auditable via `git log`
Commit-message validation	Conventional Commits enforced by `commitlint` (via lefthook)

2. Pre-commit & pre-push quality gates

EVIDENCE Developers run lint gates locally before code leaves their machine. Configured via lefthook.

`lefthook.yml`

```
commit-msg:
  commands:
    commitlint:
      run: commitlint

pre-push:
  parallel: true
  commands:
    py_lint:
      glob: "*.py"
      exclude: [".copier/*"]
      run: just py_lint {push_files}
    js_lint:
      run: just js_lint
```

3. Continuous integration & delivery

EVIDENCE Every push and pull request runs an automated pipeline in GitHub Actions. Deploys to production or staging happen *only* from protected branches after CI succeeds.

`.github/workflows/build_and_publish.yml (excerpt)`

```
on:
  push:
    branches: [master, staging]
  pull_request: {}

jobs:
  app:                       # Backend checks
    steps:
      - run: just py_setup   # install pinned Python deps
      - run: just py_lint    # Ruff + djlint
      - run: just db_migrate # apply Alembic migrations
      - run: just db_lint    # squawk-cli: safe-migration linter
      - run: just py_test    # pytest (unit + async + e2e)

  web:                       # Frontend checks
    steps:
      - run: just js_setup   # install pinned Node deps
      - run: pnpm run openapi # regenerate typed API client
      - run: just js_lint    # ESLint + Prettier + TS
      - run: just js_test    # Vitest + Testing Library

  deploy:
    needs: [web, app]
    if: github.ref == 'refs/heads/master' && github.event_name == 'push'
    environment: production   # GitHub environment protection
```

3.1 Toolchain enforced in CI

Category	Tool	Purpose
Static analysis (Python)	Ruff	Lint + import hygiene
Type checking (Python)	Pyright	Static type verification
Static analysis (frontend)	ESLint + TypeScript	Lint + types
DB migration safety	squawk-cli	Block unsafe SQL migrations
Unit / integration tests	pytest, Vitest	Regression coverage
End-to-end tests	Playwright (pytest-playwright)	Full-stack smoke tests
Coverage	coverage.py, @vitest/coverage-v8	Coverage reporting

4. Secrets management

EVIDENCE **No secrets are committed to the repository.** All credentials are stored in 1Password with separate vaults per environment (dev, staging, production, shared). Production secrets are injected into Azure Container Apps at deploy time via a 1Password service-account token.

- GitHub Actions holds only a single bootstrap secret: `OP_SERVICE_ACCOUNT_TOKEN`.
- Zoom OAuth tokens are encrypted at rest before they ever touch the database (see §5).
- Terraform reads secrets from 1Password via the `onepassword` provider, so nothing sensitive ever lives in `.tfstate` as plaintext.

5. Secure-by-default coding patterns

5.1 Encryption of OAuth tokens at rest (AES-128 / Fernet)

`app/utils/encryption.py`

```
from cryptography.fernet import Fernet
from app.configuration.encryption import ENCRYPTION_KEY
from sqlalchemy import types

_fernet = Fernet(ENCRYPTION_KEY.encode())

class EncryptedType(types.TypeDecorator[str]):
    """SQLAlchemy column type that transparently encrypts/decrypts
    string values using AES-128 (Fernet)."""

    impl = types.Text
    cache_ok = True

    def process_bind_param(self, value, dialect):
        if value is None: return None
        return _fernet.encrypt(value.encode()).decode()

    def process_result_value(self, value, dialect):
        if value is None: return None
        return _fernet.decrypt(value.encode()).decode()
```

This column type is used on every field that stores a Zoom OAuth access token or refresh token, so tokens are never written to Postgres in plaintext.

5.2 Zoom webhook authenticity & replay protection

Every inbound Zoom webhook is authenticated before the payload is parsed. The handler reads the raw body, verifies the `x-zm-signature` header against an HMAC-SHA256 computed with the app's Secret Token, and rejects any request whose `x-zm-request-timestamp` is more than 5 minutes old. Unsigned or stale requests are rejected with HTTP 401 and never reach the job queue. The URL-validation challenge (`endpoint.url_validation`) is answered only after the same signature check succeeds.

`app/routes/webhooks.py — verifier`

```
_ZOOM_SIGNATURE_TTL_SECONDS = 5 * 60

def _verify_zoom_signature(
    *, raw_body: bytes, timestamp: str | None, signature: str | None,
) -> bool:
    """Verify the x-zm-signature header Zoom sends on every webhook.
    Zoom signs each request as v0=<hex> where the HMAC-SHA256 input is
    'v0:' + x-zm-request-timestamp + ':' + raw_body keyed by the app's
    Secret Token."""
```

```
    if not ZOOM_SECRET_TOKEN:
        log.warning("ZOOM_SECRET_TOKEN unset; skipping signature verification")
        return True
    if not timestamp or not signature:
        return False
    try:
        request_ts = int(timestamp)
    except ValueError:
        return False
    if abs(int(time.time()) - request_ts) > _ZOOM_SIGNATURE_TTL_SECONDS:
        return False
    message = b"v0:" + timestamp.encode() + b":" + raw_body
    expected = "v0=" + hmac.new(
        ZOOM_SECRET_TOKEN.encode(), message, hashlib.sha256,
    ).hexdigest()
    return hmac.compare_digest(expected, signature)
```

`app/routes/webhooks.py — handler`

```
@webhook_api_app.post("/zoom")
async def zoom_webhook(request: Request):
    raw_body  = await request.body()
    timestamp = request.headers.get("x-zm-request-timestamp")
    signature = request.headers.get("x-zm-signature")

    if not _verify_zoom_signature(
        raw_body=raw_body, timestamp=timestamp, signature=signature,
    ):
        return JSONResponse({"status": "invalid signature"}, status_code=401)

    payload = json.loads(raw_body)
    if payload.get("event") == "endpoint.url_validation":
        plain_token = payload["payload"]["plainToken"]
        encrypted_token = hmac.new(
            ZOOM_SECRET_TOKEN.encode(),
            plain_token.encode(),
            hashlib.sha256,
        ).hexdigest()
        return JSONResponse(
            {"plainToken": plain_token, "encryptedToken": encrypted_token}
        )

    return await _queue_webhook(provider="zoom", request=request, payload=payload)
```

Defence in depth: the 5-minute timestamp freshness check bounds replay risk at the transport layer, while Redis-based event-ID deduplication (48 h window) in `_queue_webhook` prevents processing duplicates of legitimately signed events. Verification is covered by unit tests in `tests/routes/webhooks_test.py`.

5.3 Other controls enforced in code

- HTTPS-only for all public traffic (Azure Container Apps ingress).
- End-user authentication delegated to **Clerk** (no custom password storage).
- All Protected Health Information (PHI) is routed through the **Azure Health De-identification API** before being sent to any LLM (Azure OpenAI).
- SQLModel/SQLAlchemy parameterized queries — no string-concatenated SQL.
- Pydantic/Zod schema validation at every HTTP boundary (request & response).

6. Environment isolation

EVIDENCE Three fully separated environments, each with its own database, secrets vault, and Azure resource group:

Environment	Branch	Secrets vault	Deployed from
Development	local only	OP_DEV_VAULT_UID	developer machines
Staging	`staging`	OP_STAGING_VAULT_UID	GitHub Actions
Production	`master`	OP_PROD_VAULT_UID	GitHub Actions (environment-protected)

Infrastructure itself is versioned as code (Terraform / OpenTofu, `infra/` and `infra-staging/`), so every environment change is reviewed through the same PR process as application code.

7. Monitoring, logging & incident response

- **Sentry** — backend (`sentry-sdk[fastapi,celery,openai,sqlalchemy]`) and frontend (`@sentry/react`). Captures errors, performance traces, and unhandled exceptions, with source maps uploaded on each deploy.
- **Structured logging** — `structlog` JSON logs on the backend, `loglevel` on the frontend, aggregated centrally.
- **Health checks** — `/healthcheck` endpoint on FastAPI and dedicated Celery health worker.
- **Product analytics** — PostHog (self-hostable), used for funnel analysis, cross-referenced with Sentry.

8. Dependency hygiene

- Python: `uv.lock` pins every transitive dependency; `deptry` flags unused packages in CI.
- JavaScript: `pnpm-lock.yaml` pins every transitive dependency.
- GitHub's native security advisories / Dependabot alerts are reviewed and remediated as they surface.
- Deprecated Puppeteer / vulnerable packages surface as CI warnings and are tracked to fix.

9. Summary of SSDLC controls (auditor quick-reference)

Phase	Control	Evidence location
Plan / Design	Threat-aware design (OAuth, PHI de-ID, token encryption)	This document, §5
Develop	Pre-commit linting, typed code, parameterized queries	`lefthook.yml`, `pyproject.toml`
Build	Reproducible builds via lockfiles & Nixpacks	`uv.lock`, `pnpm-lock.yaml`, `nixpacks.toml`
Test	Automated lint/type/test gates in CI	`.github/workflows/build_and_publish.yml`

Release	Environment-protected deploys from `master` only	Same workflow, `deploy` job
Operate	Structured logs, Sentry, PostHog, health checks	Runtime; configuration in `app/`
Respond	Dependency alerts, incident triage via Sentry	GitHub Security tab, Sentry project

> **Architecture reference.** The application's data-flow diagram (submitted separately as `cosma-architecture.png`) shows exactly which components interact with Zoom and which third-party data processors handle user data.
